Data Mining Final Project Report

Group 9　312551116 黃亭幃　312551082 謝惠喻　311351009 劉承霈

---

1. Describe how you solve this problem. Details include preprocessing, embeddings, model selection, hyperparameters should be provided.

   - Data preprocessing

     由於這次資料集的news資料有非常多筆 (train: 101527, test: 72023)，並且每一個user都會需要取用數筆的新聞資料，所以為了加速訓練流程，我們會先將需要的news資料儲存成python dictionary，方便後續使用。

```python
# news embedding preprocessing
    def news_encoding(self, file_path):
        if file_path is not None:
            # 加載字典从文件
            with open(file_path, 'rb') as file:
                loaded_embeddings_dict = pickle.load(file)
            return loaded_embeddings_dict

        columns_to_keep = ['news_id', 'category', 'subcategory', 'title', 'abstract']
        df = self.news_df[columns_to_keep].copy()
        df['category'] = df['category'].astype(str)
        df['subcategory'] = df['subcategory'].astype(str)
        df['title'] = df['title'].astype(str)
        df['abstract'] = df['abstract'].astype(str)
        df['combined_text'] = df[['category', 'subcategory', 'title', 'abstract']].apply(lambda x: ' '.join(x), axis=1)

        embeddings_dict = {}
        for index, row in tqdm(df.iterrows(), total=df.shape[0], desc="Dataset news encoding"):
            news_id = row['news_id']
            combined_text = row['combined_text']
            embeddings_dict[news_id] = self.get_bert_embeddings(combined_text)

        if self.is_train:
            dict_name = 'train_news_embeddings_dict.pkl'
        else:
            dict_name = 'test_news_embeddings_dict.pkl'

        with open(dict_name, 'wb') as file:
            pickle.dump(embeddings_dict, file)

        return embeddings_dict
```

     載入news dataframe之後，會將含有文字的category、subcategory、title以及abstract四個column轉為string type後以空格連接，每筆資料會經過get_bert_embeddings轉為embedding後，存於news_id-embedding的dictionary中。

```python
self.tokenizer = RobertaTokenizer.from_pretrained('roberta-base')
self.encoder_bert = RobertaModel.from_pretrained('roberta-base')
```

```python
# Function to get BERT embeddings
    def get_bert_embeddings(self, text):
        # Tokenize input text
        inputs = self.tokenizer(text, return_tensors='pt', truncation=True, padding=True, max_length=512)

        # Get hidden states from the BERT model
        with torch.no_grad():
            outputs = self.encoder_bert(**inputs)

        # The embeddings are in the last hidden state
        embeddings = outputs.last_hidden_state

        # Mean pooling of the token embeddings to get a single vector representation
        pooled_embeddings = torch.mean(embeddings, dim=1)

        return pooled_embeddings
```

我們選擇使用hugging face提供的pre-trained RoBERTa作為encoder，將news文字encode成1×768維的embedding vector。

```python
def __getitem__(self, index):

    # history
    clicked_news = self.behaviors_df['clicked_news'][index].split()
    clicked_news_embeddings = self.get_news_list_embeddings(clicked_news)
    clicked_news_embeddings = clicked_news_embeddings.reshape(-1, clicked_news_embeddings.shape[2])
    # Padding or truncating for lstm batch training
    history_embeddings = np.zeros((self.history_seq_len, BERT_EMBEDDING_DIM))
    min_len = min(self.history_seq_len, clicked_news_embeddings.shape[0])
    history_embeddings[:min_len][:BERT_EMBEDDING_DIM] = clicked_news_embeddings[:min_len][:BERT_EMBEDDING_DIM]

    # impression
    impressions_all = self.behaviors_df['impressions'][index].split()
    impression_news = [impressions_data.split('-')[0] for impressions_data in impressions_all]
    impression_news_embeddings = self.get_news_list_embeddings(impression_news)
    impression_news_embeddings = impression_news_embeddings.reshape(-1, impression_news_embeddings.shape[2])
    if self.is_train:
        impression_labels = [float(impressions_data.split('-')[1]) for impressions_data in impressions_all]
    else:
        impression_labels = []

    return history_embeddings, impression_news_embeddings, np.array(impression_labels)
```

模型訓練會需要history_embeddings、impression_embeddings和impression_leabels三種資料，history_embeddings是由user的clicked_news取得news_id，對應回剛剛前處理的news_embedding vector，將多個embedding concatenate在一起組成的。由於需要batch training，因此我們將每個user的clicked news數量固定為35筆 (資料統計training data的平均長度為36筆，testing平均為35筆)，多的會crop掉，少於則用0 padding。impression_embeddings也是用同樣的方法取得對應的15筆news_embedding vectors。impression_labels則是training data的標註，用於supervised learning做backpropagation。

- Model

```python
class NewsRecommendationModel(nn.Module):
    def __init__(self, embedding_dim=768, hidden_dim=384, output_dim=768):
        super(NewsRecommendationModel, self).__init__()

        self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True)
        self.dropout = nn.Dropout(0.8)
        self.fc = nn.Linear(hidden_dim * MAX_CLICKED_LEN, output_dim)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x_clicked):

        lstm_out, _ = self.lstm(x_clicked)
        lstm_out = self.dropout(lstm_out)
        flat = lstm_out.reshape(lstm_out.size(0), -1)
        output = self.fc(flat)
        output = self.sigmoid(output)
        return output
```

NewsRecoomendationModel用於學習user embeddings，輸入是每個user的history_embeddings，會依序經過一層LSTM、droupout、一層linear layer，以及最後的sigmoid確保輸出在[0, 1]之間，最後的輸出會是一個768維的embedding vector。

```python
def train_one_epoch(self, train_loader, step):
    with tqdm(train_loader, unit="batch") as tepoch:
        total_loss = []
        total_auc = []
        for X_clicked_batch, X_impressions_batch, y_batch in tepoch:
            tepoch.set_description(f"Train EPOCH{step}")

            X_clicked_batch = X_clicked_batch.float().to(self.device)
            X_impressions_batch = X_impressions_batch.float().to(self.device)
            y_batch = y_batch.to(self.device)

            self.optim.zero_grad()
            output = self.model(X_clicked_batch)

            user_embeddings = output.unsqueeze(-1)  # 变为 (batch_size, 768, 1)
            dot_product = torch.matmul(X_impressions_batch, user_embeddings).squeeze(-1)  # 结果形状为 (batch_size, 15)

            loss = self.criterion(dot_product, y_batch)
            loss.backward()
            self.optim.step()
            metric = self.metrics['AUC']

            try:
                auc = metric(y_batch.cpu().detach().numpy(), dot_product.cpu().detach().numpy()).item()
            except ValueError as e:
                # Handle the case where ROC AUC cannot be calculated
                auc = 0

            tepoch.set_postfix(loss=loss.item(), auc=auc)
            total_loss.append(loss.item())
            total_auc.append(auc)
        return sum(total_loss) / len(total_loss), sum(total_auc) / len(total_auc)
```

模型的輸出會和impression計算dot product作為預測的score，再和真實標籤計算binary cross entropy loss用來更新網路權重。

- Hyperparameters
  - optimizer: Ranger21
  - loss function: BCE loss
  - learning rate: 1e-4
  - batch size: 500
  - hidden dimension: 384
  - clicked news length: 35

2. Choose a variable (e.g. different model, different approach) excluding hyperparameters and compare their performance. (You cannot change ONLY the hyperparameters) Explain what causes the difference of performance or why.

在這次的新聞推薦系統預測任務中，我們採用了不同的模型 encoder，分別是 BERT 和 RoBERTa。在相同的訓練與評估條件下，使用 RoBERTa 的 validation AUC 達到了 0.723，而 BERT 則為 0.711。

我們推測兩者在性能差異的原因如下:

1. 訓練策略上的差異: BERT 的訓練過程包含了 Next Sentence Prediction(NSP) 任務，這有助於模型學習句子間的關聯性。然而，在新聞推薦任務中，更重要的可能是對單個句子或段落內容的深入理解而不僅是句子間的關係。相比之下，RoBERTa 移除了 NSP，專注於更精細地提升 Masked Language Model (MLM) 效能，使得模型能更精確地抓住文本中關鍵詞的涵義，這對於找出新聞中的重要內容特別有幫助。

2. 資料處理與改善: RoBERTa 改進了 BERT 的 static masking 的方式，採用了 dynamic masking，在每個 epoch 中對輸入資料進行不同的遮蔽，增加了模型訓練的多樣性和複雜性。此外，RoBERTa 在訓練過程中花了更多的數據和時間，使其有更好的泛化性。

3. 模型架構與輸入處理: RoBERTa 的設計允許在訓練過程中處理更大的 batch size，這有助於模型在每個 epoch 學習到更多的資料，加速收斂速度，也意味著 RoBERTa 能更迅速適應和預測用戶對新聞的點擊行為。

3. Do some error analysis or case study. Is there anything worth mentioning while checking the mispredicted data? Share with us.

在這次的任務中，我們嘗試過使用許多方法和特徵來對是否點擊推薦新聞進行預測。在過程中遇到過許多使準確率降低的因素，我們也一一分析原因並解決。

1. 訓練資料的不平衡：在最初的訓練過程中，我們發現模型的輸出大多都是 0。在深入觀察資料以後我們發現，使用者是否點擊推薦新聞的數據極度不平衡。大多數的 label 都是 0（約佔 0.85 左右的資料）。因此，如何處理極度不平衡的資料會成為這次訓練的課題之一。我們主要嘗試了兩個方法：

1) 對 label 為 1 的資料給予更大的權重
2) 使用 Focal Loss 更加關注那些難度較大的資料

```
class FocalLoss(nn.Module):
    def __init__(self, alpha=1, gamma=2, reduction='mean'):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, inputs, targets):
        BCE_loss = F.binary_cross_entropy_with_logits(inputs, targets, reduction='none')
        pt = torch.exp(-BCE_loss)
        F_loss = self.alpha * (1 - pt) ** self.gamma * BCE_loss

        if self.reduction == 'mean':
            return F_loss.mean()
        elif self.reduction == 'sum':
            return F_loss.sum()
        else:
            return F_loss
```

使用 Focal Loss 比起第一種方法可以帶來更好的效果，使用 Focal Loss 以後，雖然在訓練初期可能會收斂的比較慢，但較可以避免模型預測大量 0 的情況發生。

2. 我們曾經嘗試過使用 ruled based method，去比較使用者點擊過的新聞（clicked news）和推薦新聞（impression news）之間的相關性，試圖去找出一個 threshold 來預測使用者是否會點擊推薦新聞。

具體的做法是這樣：我們首先將一個新聞內所具有的 special entities embedding 加總起來進行平均。得到一個長度 100 的向量作為這個新聞的代表 embedding。接著將使用者所有點擊過的新聞的代表 embedding 一樣平均起來作為這個使用者的代表

embedding。在預測階段時，我們將使用者的代表 embedding 和推薦新聞的代表 embedding 計算相似度去決定使用者是否點擊該新聞。

我們主要使用 Cosine similarity 和絕對值差來計算相似度，使用 Cosine similarity 時，訓練集 0 和 1 分別的平均相似度分別是 2.0 和 2.4，絕對值差則大約 0.8 和 1.4。但即使分別使用兩者去進行預測依據，得到的 AUC 數值也是位於 0.5 左右，可見此方法無法真正的預測使用者是否點擊新聞。

深入去觀察每筆資料的相似度會發現，其實相似度的數值差異很大。即使 label 相同的新聞，也可能因為使用者點擊新聞的數量和種類等等，造成相似度最終計算出來差距很大。因此不能直接使用此方法作為預測基準。

3. 我們在比賽過程，發現即使訓練分數很高，但丟上去 Kaggle 的結果卻常常不升反降。我們切割出 train 和 validation 以後發現，模型的 overfitting 非常嚴重，導致訓練過程的分數不能真實反映模型學習的效果。

```
Epoch 1/7: AVG Loss=1.2400318187262331, AUC=0.5060369915808274 Validation: Loss=0.6215059416634696, AUC=0.5267084834813619
Epoch 2/7: AVG Loss=2.73215583765081, AUC=0.5116777782370951 Validation: Loss=0.42429699642317636, AUC=0.5347213358583142
Epoch 3/7: AVG Loss=2.3232504619019374, AUC=0.5187766701597007 Validation: Loss=0.5455534287861415, AUC=0.4861019377158641
Epoch 4/7: AVG Loss=0.9488634493734155, AUC=0.5237522479867981 Validation: Loss=0.4832104742527008, AUC=0.6061834760267905
Epoch 5/7: AVG Loss=1.0685641595295496, AUC=0.5610637805590033 Validation: Loss=0.3730950951576233, AUC=0.5961834123267912
Epoch 6/7: AVG Loss=0.7932948773460728, AUC=0.5743731938611752 Validation: Loss=0.4374229056494577, AUC=0.5931924512262901
Epoch 7/7: AVG Loss=0.40803216823509764, AUC=0.5892102357097555 Validation: Loss=0.3437058563743319, AUC=0.576481212211021
```

因此我們使用了幾種方法來防止 overfitting 的發生：

1) 在模型上使用 Dropout 技術。
2) 將模型的複雜度調低。
3) 切割 train 和 valiationd 資料集追蹤 overfitting 的發生，並使用 validation performance 進行 save best 保存最好的模型。

```
Epoch 1/7: AVG Loss=1.0345413117294768, AUC=0.5480596009751615 Validation: Loss=2.631588007722582, AUC=0.5642892403400246
Epoch 2/7: AVG Loss=1.1766487705517574, AUC=0.5617525125696218 Validation: Loss=0.37526485323905945, AUC=0.5917021144875625
Epoch 3/7: AVG Loss=0.7028822471421078, AUC=0.5743185116809805 Validation: Loss=0.7989980280399323, AUC=0.5734756999289999
Epoch 4/7: AVG Loss=0.4968094296189418, AUC=0.5934755607886354 Validation: Loss=0.36344339485679356, AUC=0.59457864377979
Epoch 5/7: AVG Loss=0.7197914840690643, AUC=0.5948448078126853 Validation: Loss=0.35777138067143305, AUC=0.6353465258707531
Epoch 6/7: AVG Loss=0.41888180660061625, AUC=0.6228233057597564 Validation: Loss=0.3739933116095407, AUC=0.5934847845029105
Epoch 7/7: AVG Loss=0.3407211307272968, AUC=0.6660658122161124 Validation: Loss=0.3257251871483667, AUC=0.682185768646144
```

可以看到套用以上方法以後大幅減少了 overfitting 的情形。